

# 基于神经网络的混凝土抗压强度预测

姓名：李垠莹

学号：SA25219012

日期：2026年3月23日

```
In [ ]: import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split

plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'SimSun']
plt.rcParams['axes.unicode_minus'] = False
```

## 1 实验目的

本实验旨在利用神经网络方混凝土凝土抗压强度进行预测，掌握回归问题中数据预处理、特征分析、神经网络建模、模型训练与测试评估的基本流程。通过本实验，进一步理解神经网络在非线性回归问题中的应用。

## 2 数据集介绍

本实验采用混凝土抗压强度数据集，共包含1030个样本。每个样本由8个输入特征和1个输出标签组成。输出混凝土抗压强度。

## 3 实验方法

### 3.1 数据集预处理

本实验初始使用前80%样本作为训练集、后20%样本作为测试集。后续进行了调整。

```
In [ ]: # 读取数据并划分训练集与测试集
def load_data(csv_path):
    df = pd.read_csv(csv_path)
    # 检查是否有数据缺失
    print("Missing values in each column:")
    print(df.isnull().sum())
```

```

# 前8列作为输入，最后1列作为输出
X = df.iloc[:, :-1].values.astype(np.float32)
y = df.iloc[:, -1].values.astype(np.float32).reshape(-1, 1)

# 划分
n = len(df)
split_idx = int(0.8 * n)
X_train = X[:split_idx]
y_train = y[:split_idx]
X_test = X[split_idx:]
y_test = y[split_idx:]

# 只用训练集进行训练
scaler_X = StandardScaler()
X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)

return df, X_train, y_train, X_test, y_test

```

## 3.2 神经网络模型构建

我实验采用前馈神经网络进行回归预测。网络输入层包含8个神经元，对应8个输入特征；输出层包含1个神经元，用于输出预测的混凝土抗压强度；中间设置两层层，隐藏第一层隐藏层有64个神经元，第二层有32神经元层，并采用ReLU激活函数增强模型的非线性表达能力。

```

In [ ]: class ConcreteNet(nn.Module):
    def __init__(self, input_dim=8, activation_name="relu", dropout=0.0):
        super().__init__()
        act1 = get_activation(activation_name)
        act2 = get_activation(activation_name)

        layers = [
            nn.Linear(input_dim, 64),
            act1
        ]
        if dropout > 0:
            layers.append(nn.Dropout(dropout))

        layers.extend([
            nn.Linear(64, 32),
            act2
        ])
        if dropout > 0:
            layers.append(nn.Dropout(dropout))

        layers.append(nn.Linear(32, 1))
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)

```

## 3.3 训练流程与测试指标

训练阶段使用训练集对模型进行训练优化，使用均方误差损失函数（MSELoss），优化器为 Adam。训练过程中记录每轮训练损失变化，观察模型收敛情况。

训练完成后，使用测评估模型性能评估，并计算均方误差（MSE）、均方根误差（RMSE）、平均绝对误差（MAE）以及决定系数（ $R^2$ ）等指标。

```
In [1]: def train_model(model, train_loader, criterion, optimizer, epochs=300, device="cpu"):
    model.to(device)
    train_losses = []

    for epoch in range(epochs):
        model.train()
        epoch_loss = 0.0

        for batch_x, batch_y in train_loader:
            batch_x = batch_x.to(device)
            batch_y = batch_y.to(device)

            pred = model(batch_x)
            loss = criterion(pred, batch_y)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item() * batch_x.size(0)

        epoch_loss /= len(train_loader.dataset)
        train_losses.append(epoch_loss)

        if (epoch + 1) % 20 == 0:
            print(f"Epoch [{epoch+1}/{epochs}], Train Loss: {epoch_loss:.4f}")

    return train_losses
```

```
In [ ]: def evaluate_model(model, X_test, y_test, device="cpu"):
    model.eval()
    model.to(device)

    with torch.no_grad():
        X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
        pred = model(X_test_tensor).cpu().numpy()

    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, pred)
    r2 = r2_score(y_test, pred)

    return pred, mse, rmse, mae, r2
```

## 3.4 相关性分析

最初没有进行相关性分析时，进行了300轮训练，发现虽然 Loss 值降至 17 左右，但是  $R^2 = -0.0808$ ，模型对测试集的泛化能力很差，推测是训练集与测试集之间的部分特征分布存在差别，为了为分析各输入特征与混凝土抗压强度之间的关系，先计算相关系数矩阵，绘制特征值与预测目标的相关性条形图。

```
In [ ]: def correlation_analysis(df, save_dir):  
    corr = df.corr(numeric_only=True)  
  
    print("\n与目标 csMPa 的相关性: ")  
    print(corr["csMPa"].sort_values(ascending=False))  
  
    # 各特征与目标相关性条形图  
    target_corr = corr["csMPa"].drop("csMPa").sort_values(ascending=False)  
  
    plt.figure(figsize=(6, 5))  
    target_corr.plot(kind="bar")  
    plt.ylabel("相关系数")  
    plt.title("各特征与混凝土强度的相关性")  
    plt.grid(True, axis="y")  
    plt.tight_layout()  
    plt.savefig(os.path.join(save_dir, "feature_target_correlation.png"), dpi=300)  
    plt.show()
```

## 4. 结果分析

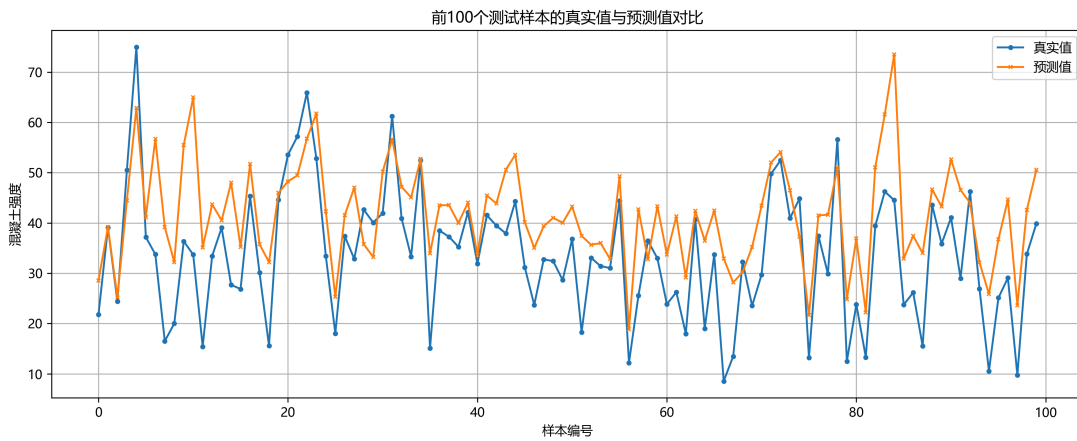
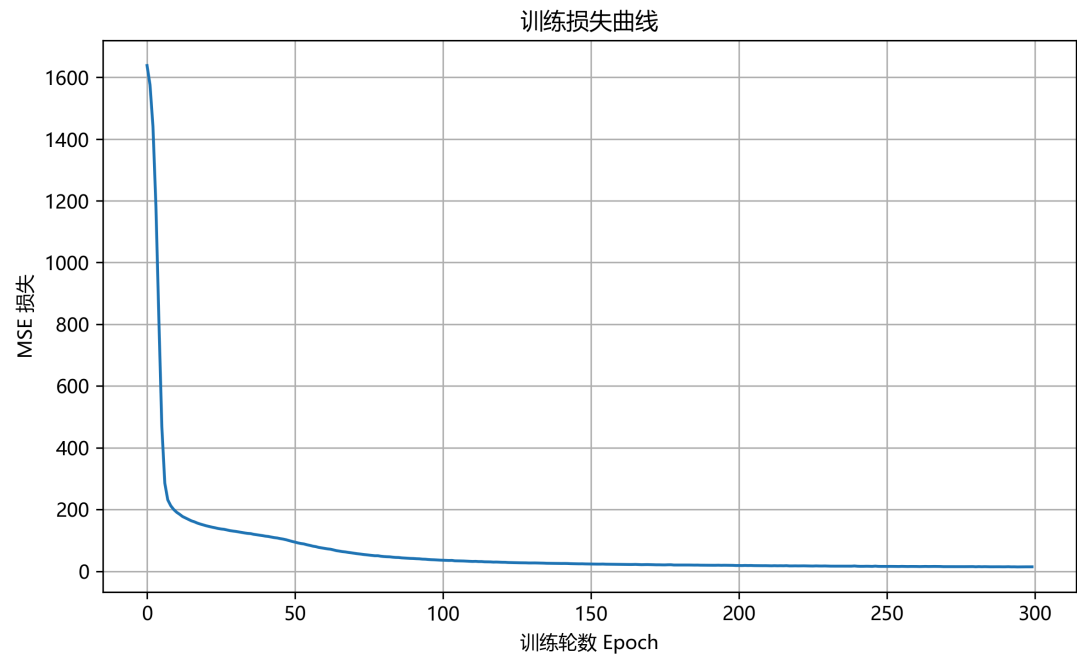
### 4.1 初步拟合结果分析

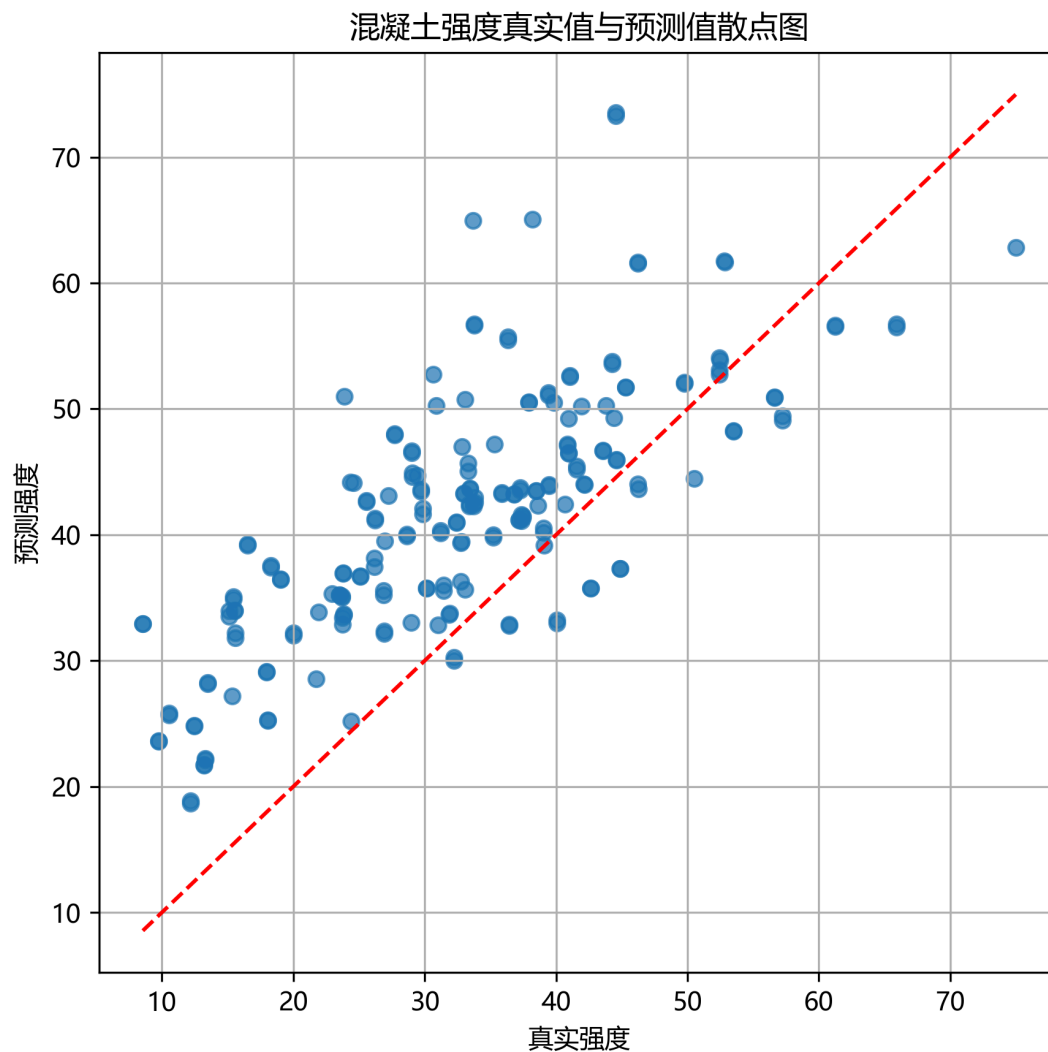
从训练损失曲线可以看出，模型有在逐步优化，loss 逐步下降并逐渐收敛到比较低的值，模型对训练数据集的拟合是在逐步改善的。

从测试集评价指标来看，模型在测试集上的表现为：

- MSE = 112.1836
- RMSE = 10.5917
- MAE = 8.8198
- $R^2 = 0.2650$

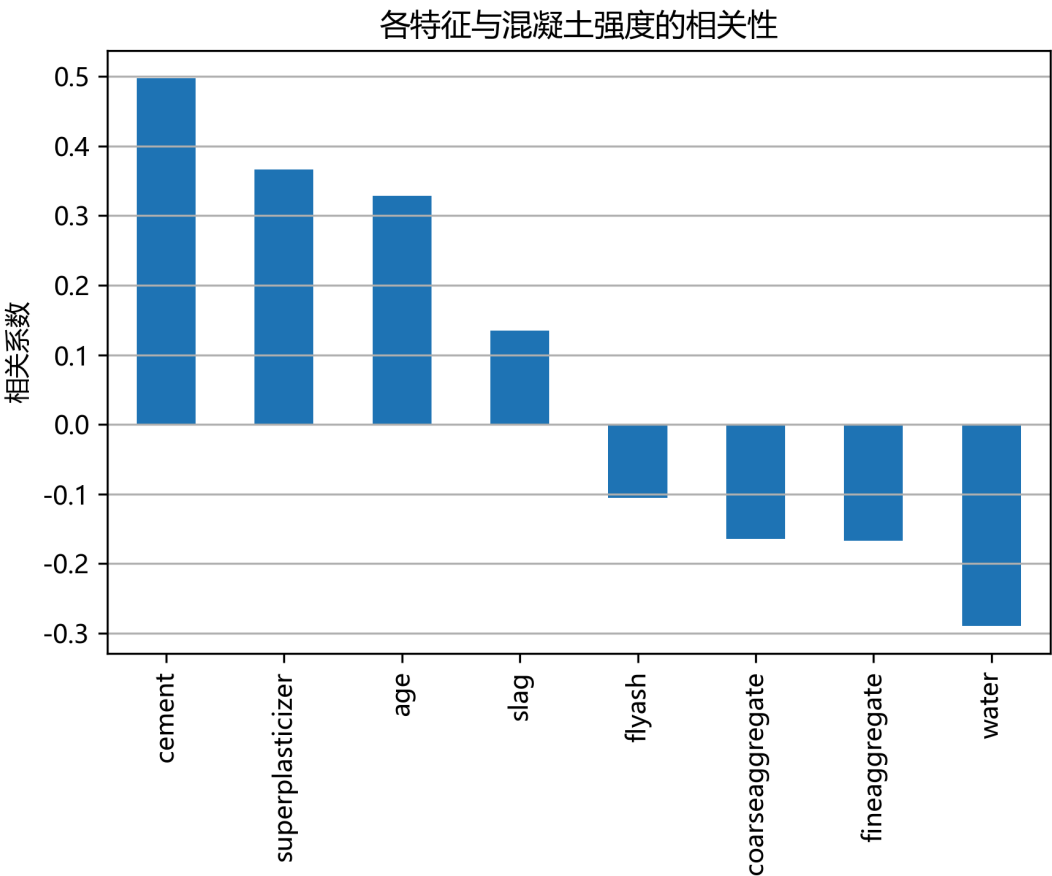
对测试集的样本进行测试并绘制了 100 个样本的预测值与真实值对比图，从对比图可以看出，橙色预测曲线随着蓝色的真实曲线在波动，波动的升降趋势大部分是一致的，但是预测值与真实值仍然存在差异，对极端样本拟合不够好，出现“向均值收缩”的情况。





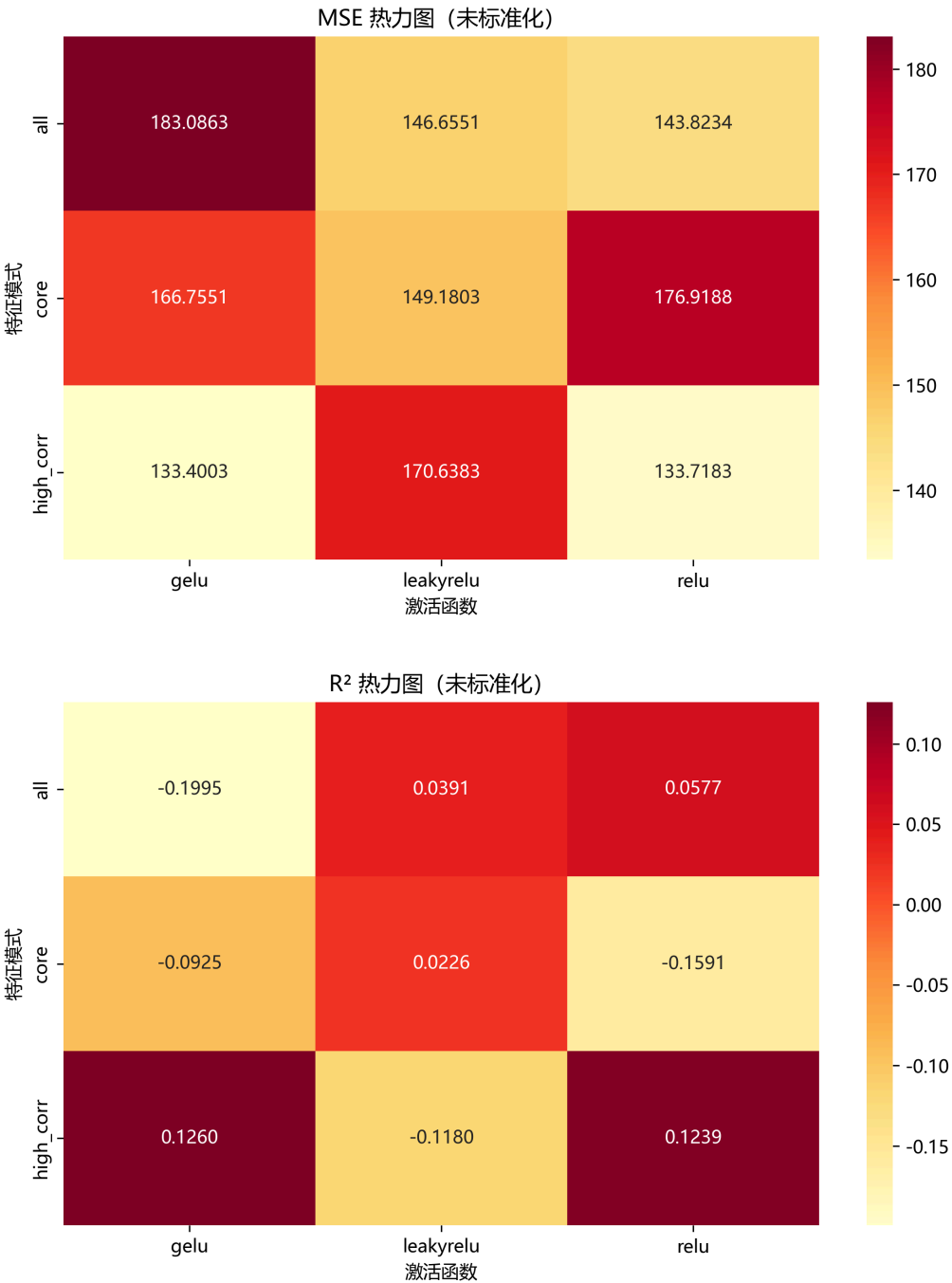
## 4.2 相关性分析

由相关性分析可知，cement、superplasticizer 和 age 这几个特征与混凝土抗压强度具有比较明显的正相关，而 water 与强度呈负相关，说明这四个特征对模型预测起到比较重要的作用。



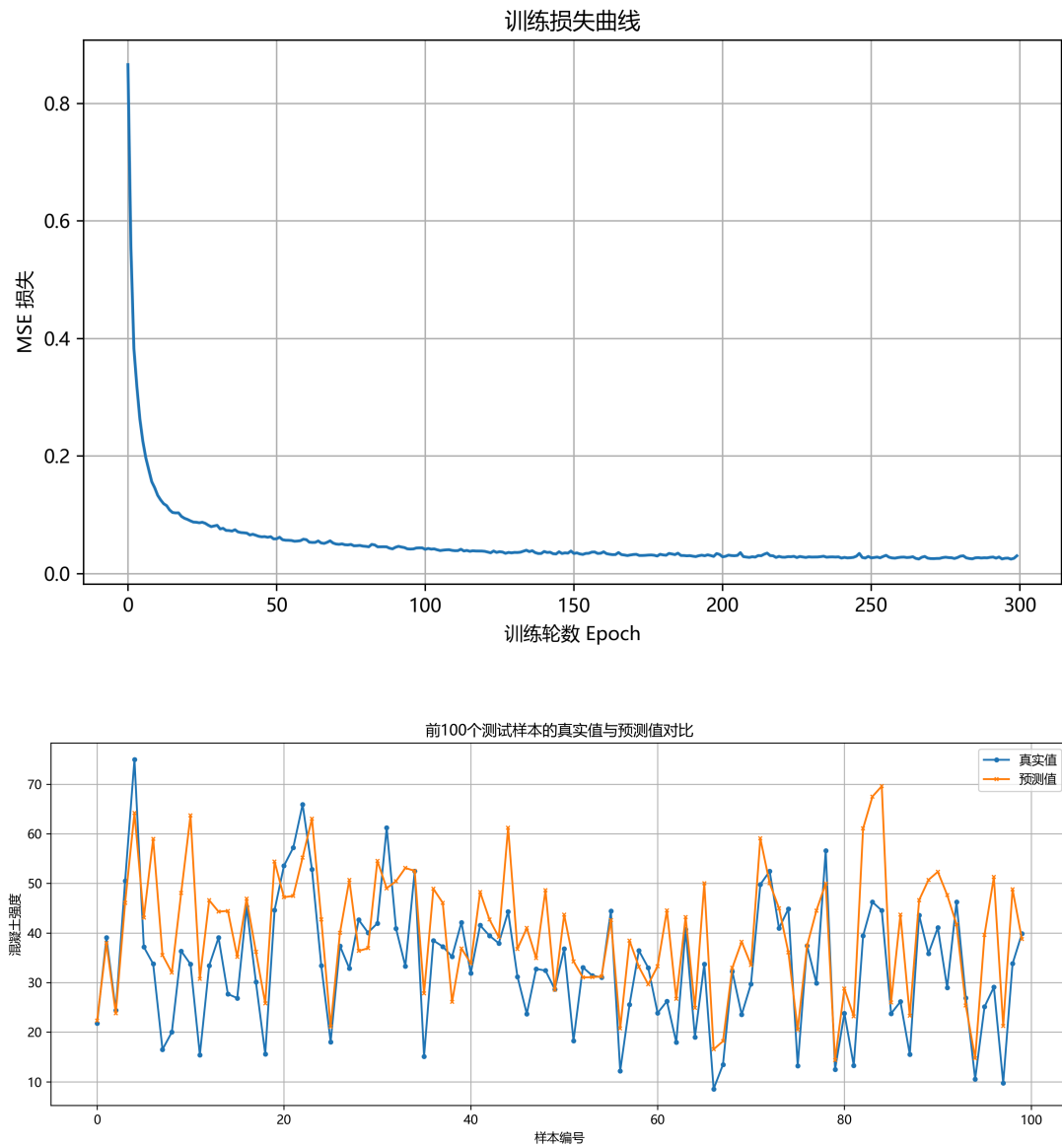
4.3 拟合结果优化

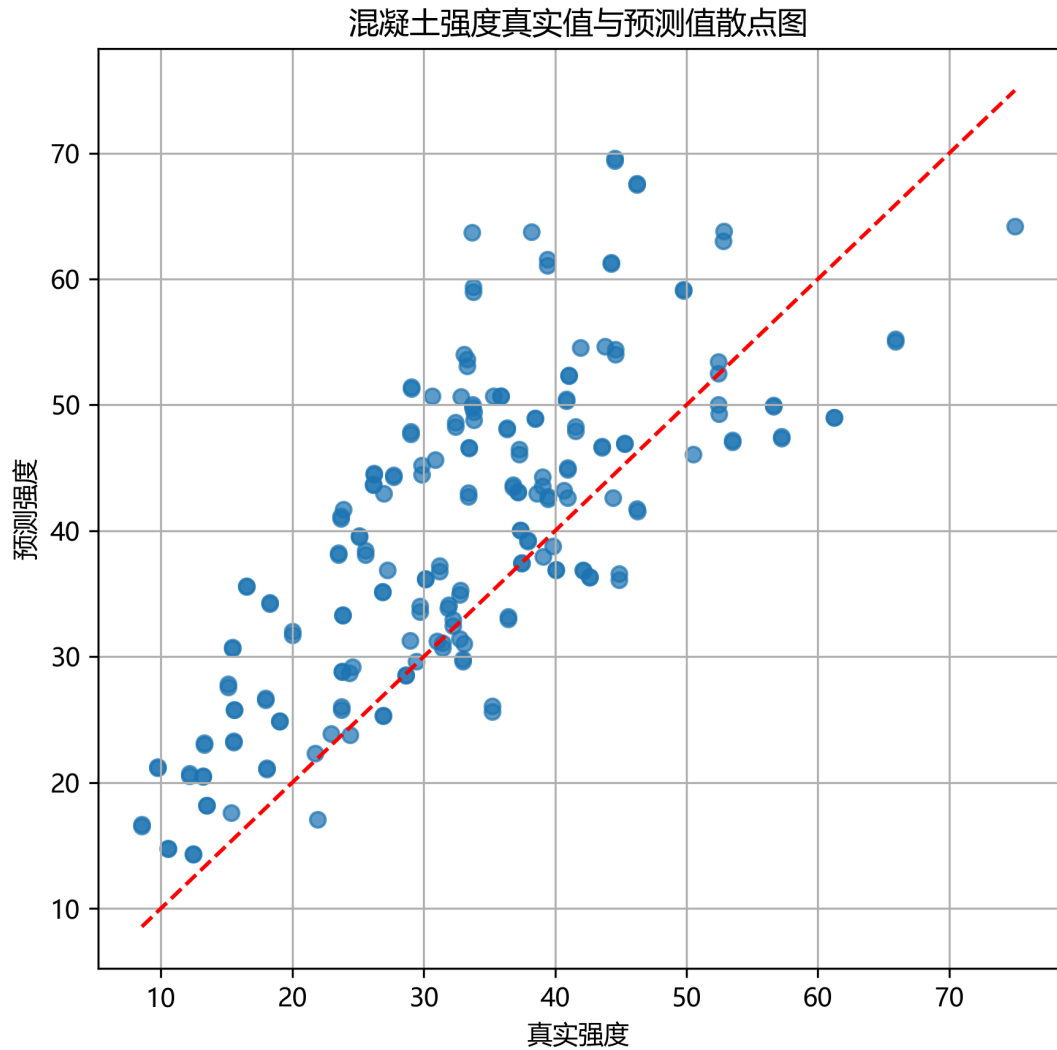
在进行了相关性分析后，为了进一步提升拟合精度，尝试只使用部分主要特征进行拟合，同时尝试使用其它激活函数，进一步优化。



从实验结果可见，使用全部 8 个输入特征时，模型整体表现优于仅使用核心特征的方案。从特征选择角度看，仅保留高相关特征的方案具有一定可行性，high\_corr + gelu 和 high\_corr + relu 的 MSE 分别为 133.4003、133.7183，整体表现优于部分较弱方案，说明基于相关性分析进行特征筛选是有意义的。但如果进一步缩减到 4 个核心特征，模型性能会整体下降，说明过度压缩输入特征会导致信息损失，从而削弱模型预测能力。单变量相关性高并不意味着其余低相关特征一定无用，在多变量非线性模型中，部分弱相关特征仍可能提供补充信息。在激活函数方面，ReLU 的表现最稳定，LeakyReLU 次之，而 GELU 在当前实验设置下效果较差；此外，我还尝试了对目标值 y 进行标准化，使模型的测试集 MSE 从 143.8234 下降到 127.6525，R<sup>2</sup> 从 0.0577 提升到 0.1636，得到了有效的提升。使用全部输入特征及 ReLU 激活函数，同时对 y 标准化后的结果如下图所示：







#### 4.4 更改数据集划分策略

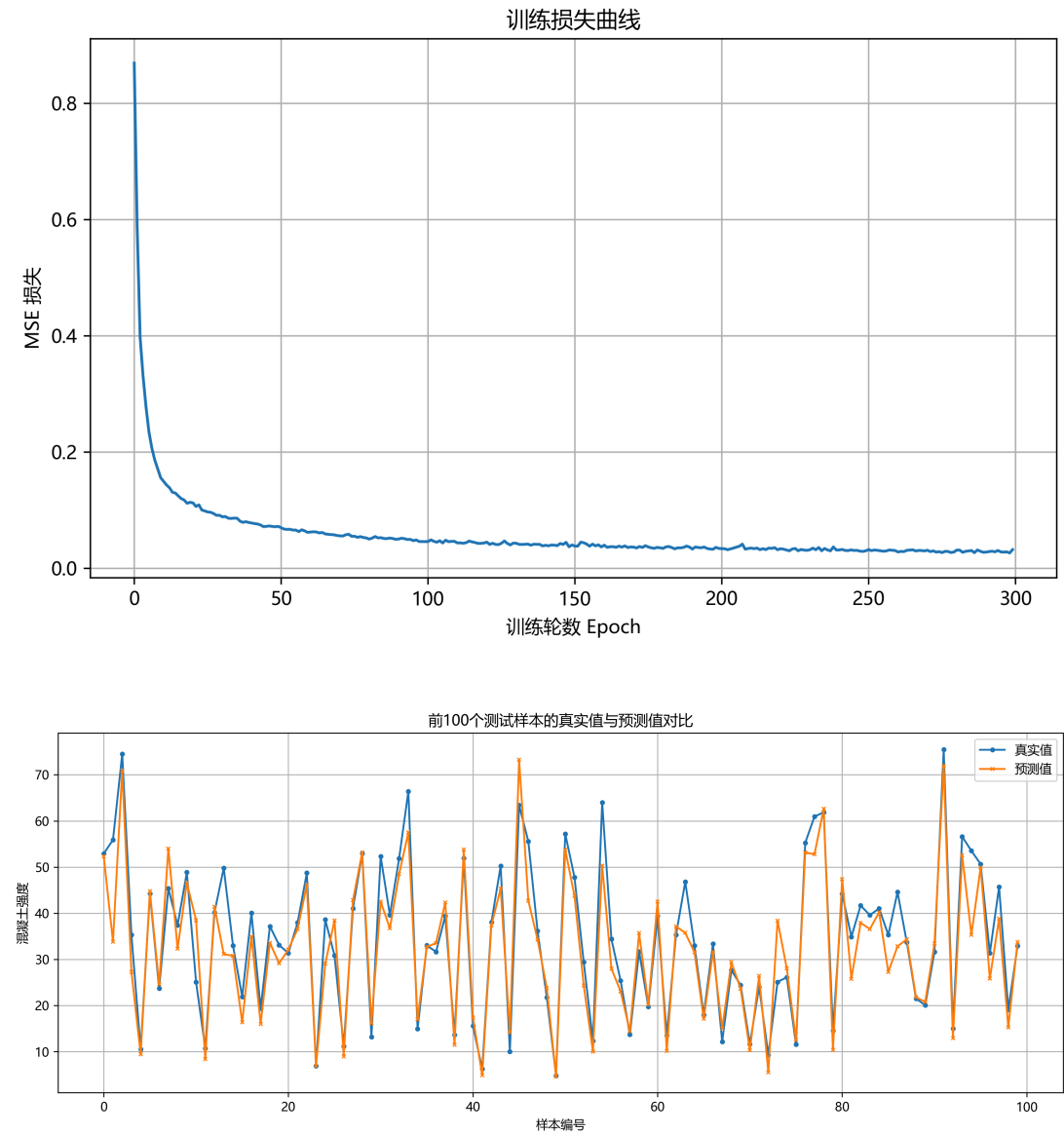
由于  $R^2$  极低，我查看了数据集的分布，发现采用前后划分时，训练集和测试集在 `age` 等特征上的分布很不一致，从而导致模型难以从训练数据集泛化到测试数据集，所以我尝试了随机划分，在数据集中随机抽取 80% 的样本作为训练数据集，剩下的样本作为测试数据集。

采用全部特征作为输入，激活函数为ReLU，训练300轮后，模型在测试集上的表现为：

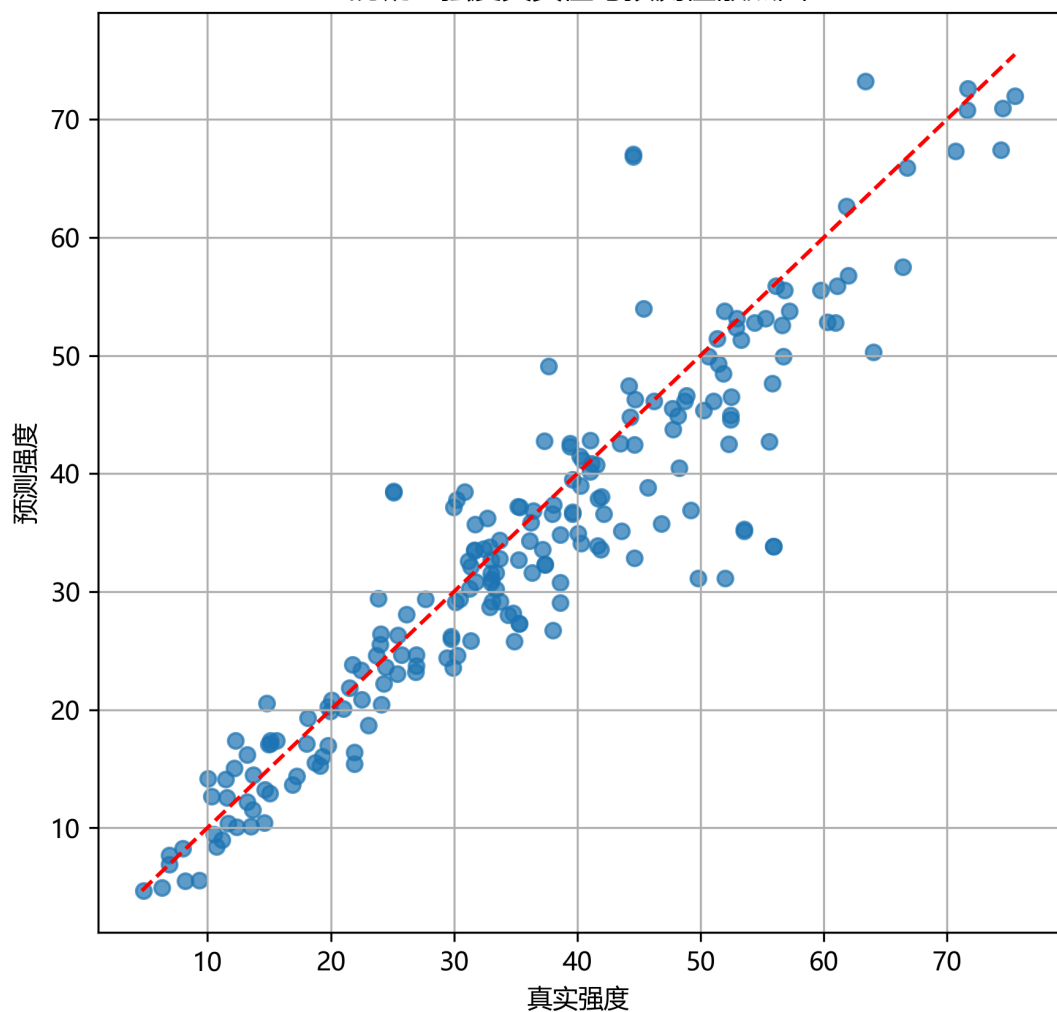
- $MSE = 38.2338$
- $RMSE = 6.1833$
- $MAE = 4.2629$
- $R^2 = 0.8516$

模型的预测精度显著提升了，说明在随机划分下，模型已经能够较好地学习混凝土配方与抗压强度之间的关系，并且在测试集上也有很强的泛化能力。证明之前的划分方式，由于

训练集和测试集分布差异太大，导致模型无法很好地发挥性能。



混凝土强度真实值与预测值散点图



```
In [ ]: def load_data_random(csv_path, feature_mode="all", use_y_scaling=False, test_size=0.2):
    df = pd.read_csv(csv_path)

    print("Missing values in each column:")
    print(df.isnull().sum())

    selected_cols = get_selected_features(feature_mode)

    X = df[selected_cols].values.astype(np.float32)
    y = df["csMPa"].values.astype(np.float32).reshape(-1, 1)

    # 随机划分
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=test_size,
        random_state=random_state,
        shuffle=True
    )

    scaler_X = StandardScaler()
    X_train = scaler_X.fit_transform(X_train)
    X_test = scaler_X.transform(X_test)

    scaler_y = None
```

```
if use_y_scaling:
    scaler_y = StandardScaler()
    y_train_scaled = scaler_y.fit_transform(y_train)
    y_test_scaled = scaler_y.transform(y_test)
else:
    y_train_scaled = y_train
    y_test_scaled = y_test

return (
    df,
    selected_cols,
    X_train, y_train, y_train_scaled,
    X_test, y_test, y_test_scaled,
    scaler_X, scaler_y
)
```

## 5 总结

本实验较完整地实现了回归分析的基本流程。实验以混凝土抗压强度数据集为对象，围绕神经网络回归预测任务开展了模型设计与对比分析。实验使用 8 个输入特征和 1 个输出变量，使用前 80% 样本作为训练集、后 20% 样本作为测试集进行建模，并以均方误差作为主要评价指标。与此同时，通过改变激活函数和主要特征选择，进一步设计了多组对比实验，对特征组合、激活函数以及目标值标准化策略进行了系统比较。